Project: TimeTracker

An AI-Driven Full-Stack Software Development Project

Domain: ***AI-Driven Software Engineering – Full-Stack Web Development (Kotlin/Spring Boot & React/TypeScript) and Agentic Coding Tools***

Work carried out by: **Ayoub Alila**

GitHub: **ayoubalila** *Matriculation No.:* **118766**

Repository: `https://github.com/se2p-classrooms/final-project-ayoubalila`

Academic year: summer semester 2026

Note on the Use of Artificial Intelligence

In the context of this project, AI tools were used deliberately and transparently, in line with the AI-driven nature of the assignment itself. The agentic coding tool **Claude Code** (Anthropic) was used from the terminal and through its VS Code integration for repository exploration, planning support, full-stack implementation, test writing, CI configuration and debugging, and documentation. **Perplexity** was used as a conversational support tool outside the repository, to clarify assignment requirements, interpret project status and CI results, and prepare focused, repository-aware prompts.

All AI-generated code and documentation changes were reviewed by me before being accepted, and were validated through unit, integration, frontend, mutation, and system tests together with the GitHub Actions pipeline. AI tools served as an implementation and reasoning accelerator, not as an autonomous replacement for engineering judgement. AI assistance was also used punctually to improve the wording and readability of parts of this report; the analysis, reflections, and conclusions are my own.

Contents

1 Introduction

The rapid maturation of large language models (LLMs) and agentic coding tools is transforming how software is designed, implemented, and verified. Rather than acting only as autocomplete assistants, modern tools such as Claude Code can read an entire repository, plan multi-step changes, write and run tests, and iterate on CI failures. This shift raises a central engineering question: how should a developer structure their workflow so that AI acceleration does not come at the cost of architectural coherence, correctness, or maintainability?

This report addresses that question through the development of **TimeTracker**, a full-stack time-tracking application built as part of an AI-driven software development lab. TimeTracker consists of a React/TypeScript single-page frontend, a Kotlin/Spring Boot REST backend, a PostgreSQL database with Flyway-managed migrations, and JWT-based authentication, all integrated and verified through a comprehensive automated test suite and a GitHub Actions CI pipeline.

The report reflects on my individual contribution to the project and on my use of AI-based tools throughout the development process. It follows the structure recommended in the assignment specification, preceded by an account of my own role and methodology, and followed by a dedicated section on the three custom features, a discussion, and a conclusion with perspectives for future work.

2 My Role and Development Methodology

My contribution was to drive the architecture, define and prioritise the milestones, choose the technology stack, formulate repository-aware prompts, review AI-generated output, validate changes through tests and CI, and integrate all work into one coherent application. AI tools were an implementation and reasoning accelerator, not an autonomous replacement for engineering judgement.

The workflow I followed was consistently iterative:

1. I translated the assignment into a milestone-based plan (`PLANNING.md`, `TASKS.md`) with issues, acceptance criteria, and dependencies between milestones.
2. For each feature, I asked Claude Code to first read the relevant repository documentation and the current implementation before writing anything, so that generated code matched the existing architecture rather than an assumed one.
3. I gave focused tasks scoped to one issue or milestone at a time, rather than requesting the whole system in one prompt.
4. I reviewed the resulting code and documentation changes myself before accepting them.
5. I verified results with unit, integration, frontend, mutation, and system tests, together with the GitHub Actions pipeline, rather than trusting that generated code compiled or behaved as intended.
6. When failures occurred, I used the exact test output or CI logs to narrow the next prompt and to request a minimal, evidence-based fix instead of a broad rewrite.

This structured use of AI, decomposing work, constraining scope, and closing the loop with automated verification, was itself one of the main learning outcomes of the lab, arguably as significant as any individual feature implemented.

3 System Architecture and Design Decisions

TimeTracker is a full-stack time-tracking application: a React/TypeScript single-page frontend talking to a Kotlin/Spring Boot REST backend backed by PostgreSQL, with JWT-based authentication and Flyway-managed schema migrations. The backend follows a conventional layered structure (controller → service → repository), with DTOs kept separate from JPA entities so entities are never exposed directly over REST.

Frontend. React, TypeScript, and Vite were chosen for a typed, component-based frontend with a mature ecosystem and fast local iteration (HMR), which mattered given the compressed timeline. I considered alternatives such as Vue or Angular in principle, but did not build prototypes or run a formal comparison, the decision was driven by React's fit for the interactive, stateful UI (live timer, nested project trees, modals) and by the availability of mature documentation, established libraries, and a broad ecosystem. shadcn/ui and Tailwind CSS were used for accessible, unstyled-by-default components without a heavy runtime, which kept the UI consistent as it grew.

Backend. The JVM is mandated by the assignment, which fixed the language family; Kotlin was chosen over plain Java for its data classes and null-safety, which reduce boilerplate in DTOs and entities. Spring Boot was chosen for its mature, well-documented REST, validation, security, and JPA integration, all of which are also well represented in LLM training data.

Persistence. PostgreSQL was chosen for the relational domain, users, hierarchical projects, project memberships, tasks, tags, budgets, and rates, and specifically for its support of recursive common table expressions, which are used to aggregate tracked time across a project and all of its descendants exactly once. H2 is used only for fast in-memory unit/integration tests so the required test suite does not depend on Docker. Flyway provides versioned, reproducible migrations (V1–V9, one per schema change, see `PLANNING.md`); Spring Data JPA handles ordinary CRUD access, while the hierarchical aggregation query is implemented as raw SQL because it is not naturally expressible through the repository abstraction.

Authentication. I chose stateless JWTs over server-side sessions because the frontend and backend are deployed as separate processes and token-based authentication keeps the API simple for the assignment scope. Persistence of a running task across browser restarts is handled by storing the task start time in the database, while the frontend recalculates the elapsed time after login. I weighed this against session-based authentication in principle but did not prototype it, since JWTs were the more natural fit for the chosen client/server split.

CI/CD evolution. `PLANNING.md` §7 originally sketched a straightforward three-job pipeline (backend, frontend, system tests) running commands directly on the runner. Building out the System (Playwright) job showed that end-to-end tests needed a properly isolated PostgreSQL instance, a matching JDK/Node toolchain, and a browser build that all agreed with one another, none of which the simple sketch accounted for. The project moved to a self-hosted runner with every job (backend, frontend, system) running inside a pinned container image (Issue #25, PR #26); the System job specifically combines a GitHub Actions PostgreSQL service container with host-installed JDK/Node tooling and the official Playwright browser image. Reaching that configuration took more iterations than expected, discussed in §5 and §6.

4 Used LLMs and AI-Based Tools

The main implementation tool was **Claude Code**, an agentic coding CLI used from the terminal (and its VS Code integration) for repository exploration, planning support, full-stack implementation, writing tests, CI configuration and debugging, documentation, and iterative UI verification via ad-hoc screenshot scripts. I do not name a specific underlying model version, since the session-to-session model was not something I tracked or can reliably reconstruct.

I also used **Perplexity** as a conversational support tool outside the repository: to understand and clarify assignment requirements, to interpret project status and CI results at a higher level, to plan the next milestone, and to prepare focused, repository-aware prompts before handing a task to Claude Code.

`PLANNING.md`, `TASKS.md`, and `CLAUDE.md` were maintained iteratively with AI assistance and my own review throughout the project, rather than written once at the start. These files acted as persistent, repository-resident instructions for the agent: they encode the architecture, coding conventions, security constraints (e.g. the project-membership access rule), feature dependencies between milestones, and testing expectations, so that each new prompt did not have to restate this context from scratch. Configuring the agent this way, through durable project files rather than one-off chat messages, was as important to the workflow as any individual prompt.

5 Where AI Tools Worked Well

Incremental full-stack delivery. Claude Code was most effective when the project was decomposed into small, clearly scoped milestones with acceptance criteria already written down. Within that structure, it accelerated consistent implementation across Kotlin/Spring Boot services, DTOs, validation, and REST endpoints, together with the matching React components, API integration, and tests, for example the project-sharing milestone (Issue #41–Issue #46) and the billable-rates feature (Issue #71–Issue #75, PR #76). I retained control by reviewing each change against the existing architecture and by requiring the accompanying unit and integration tests to pass before moving on.

Quality assurance and test hardening. AI assistance was useful for locating untested branches and proposing targeted tests to close coverage gaps once both the backend (Jacoco) and frontend (Vitest) gates were set to a 90% line/branch/function/statement threshold, and for the Pitest mutation-testing baseline on the service layer (Issue #39, Issue #40). It worked best when I gave it a concrete coverage report or a specific failing/surviving case, rather than a vague instruction to "improve testing."

Final Playwright CI diagnosis. The resolution of the System (Playwright) CI failure illustrates AI working well once given the right constraints. After I supplied the exact CI error text, restricted the task explicitly to the failing job, and required inspection of the actual workflow file before any edit, Claude Code correctly identified that the Playwright container ran as root while GitHub Actions forced `HOME=/github/home` (owned by a different user inside that image), which is exactly the condition under which Firefox's sandbox refuses to launch. It proposed the minimal, job-scoped `HOME=/root` fix documented in Firefox's own error output (`f616cb8`), reasoning explicitly about why the fix had to be job-scoped rather than step-scoped to avoid invalidating the Gradle/npm caches used by the same job. The System job subsequently passed.

6 Where AI Tools Did Not Work as Intended

The same CI story is also the clearest example of AI underperforming. Stabilising the System (Playwright) job took a chain of separate fix commits (`748979c`, `2487617`, `caa9ec4`, `774cff6`, `58c2f67`, `d2ecb98`, `f616cb8`) introduced after the E2E suite itself (Issue #79, PR #80). Early fixes were too local: each addressed the symptom visible in the last failed run, missing Docker CLI, service-container networking, native-runner Postgres access, missing JDK/Node setup, a browser version mismatch, without accounting for how the self-hosted runner, Docker, PostgreSQL, Node tooling, Playwright's bundled browsers, container permissions, and Firefox's own sandboxing all interacted. A plausible, locally-correct patch was repeatedly not a complete fix for the environment as a whole.

I recognised this pattern through the next CI run: each "fixed" commit reliably exposed a different, previously hidden assumption rather than a full resolution. In response, I adapted from broad prompts such as "fix CI" to evidence-rich requests that included the exact failing log, named the single job to change, explicitly asked for root-cause reasoning before any edit, and required the smallest change that would not regress the backend or frontend jobs, which were already green. That shift in prompting is what ultimately produced the correct, minimal, job-scoped diagnosis described in §5. The broader lesson was that AI is most reliable when grounded in the actual repository state and concrete execution evidence; it can meaningfully accelerate diagnosis, but it does not remove the need for engineering judgement, verification against real logs, and iterative debugging.

7 Missing AI Tools

A tool I would have found valuable but did not have access to is one that can inspect and reproduce the *actual* CI runtime environment, container image, user/UID, filesystem permissions, mounted directories, installed dependencies, service containers, browser runtime configuration, and environment variables, and then test a proposed workflow fix inside an equivalent sandbox before it is pushed to the real pipeline. During the Playwright/Firefox debugging described in §6, every hypothesis had to be validated by pushing a commit and waiting for a full CI run; a local or sandboxed replica of the self-hosted runner's container setup would have shortened that trial-and-error loop considerably.

8 Custom Features and Design Rationale

Beyond the mandatory Part I/II requirements, three custom features were designed and implemented, each extending the base time-tracking model rather than sitting alongside it as an unrelated addition.

8.1 Task Tags with Cross-Cutting Filters

Project hierarchy alone cannot represent every way a user wants to categorise work: a task can belong to one project, but a tag can cut across projects (e.g. "urgent", "billable-review"). Tags let a user classify tasks independently of the project tree and filter the dashboard, project task views, and CSV exports by tag. A deliberate design choice was to scope tags per user rather than per project: tags are personal organisation, so on a shared project a collaborator's own tags are never exposed to or reused by other members, keeping the sharing model (project data shared, personal categorisation private) consistent.

Migration: Issue #58. *Feature issues:* Issue #59, Issue #60, Issue #61, Issue #62. *Tests:* Issue #63. *Implementation:* PR #64.

8.2 Project Time Budgets with Progress Alerts

Budgets turn time tracking from a purely retrospective log into a planning aid. Supporting total, weekly, and monthly budget periods covers both fixed-scope work (a total budget for a project) and recurring work patterns (a weekly or monthly allowance). Rather than only reporting a historical percentage on a dashboard, the threshold warning (yellow at $\geq 80\%$, red at $\geq 100\%$) is deliberately surfaced right after a task is stopped, i.e. at the moment a user's own action changes the budget usage, so the feedback is actionable rather than something discovered later while browsing a report. For shared projects, budget usage is computed over all members' combined time, consistent with how shared-project totals are already computed elsewhere in the application.

Migration: Issue #65. *Feature issues:* Issue #66, Issue #67, Issue #68. *Tests:* Issue #69. *Implementation:* PR #70.

8.3 Billable Rates and Cost Reporting

Billable rates extend the application from time recording into cost reporting, which is directly useful for the freelancer/researcher personas the application targets. An hourly rate can be set on any project; if a (sub-)project has no rate of its own, it inherits the nearest ancestor's rate by walking up the project tree, which avoids having to configure a rate on every subproject individually while still allowing an explicit override lower in the hierarchy. The resolved ("effective") rate and the resulting cost are surfaced at task level, at project level, and as additional columns in the CSV export, so the same cost figures are available both inside the UI and in external spreadsheets.

Migration: Issue #71. *Feature issues:* Issue #72, Issue #73, Issue #74. *Tests:* Issue #75. *Implementation:* PR #76.

9 Discussion

Taken together, the experiences documented in this report converge on one central observation: the value of AI-driven development depends far less on the raw capability of the model than on the *engineering discipline wrapped around it.*

Where the workflow was structured, milestones with written acceptance criteria, repository-resident context files (`PLANNING.md`, `TASKS.md`, `CLAUDE.md`), one issue per prompt, and mandatory verification through tests and CI, Claude Code delivered consistent, architecture-conforming code across the full stack at a pace I could not have matched alone. Where the structure was weakest, most visibly in the early CI debugging phase, the same tool produced a chain of plausible but locally-scoped patches, each of which resolved one visible symptom while missing the interaction between the runner, containers, tooling, and browser sandboxing.

Three broader points follow from this contrast. First, AI tools amplify the quality of their input: precise, evidence-rich prompts grounded in real logs and repository state yielded root-cause fixes, while broad prompts such as "fix CI" yielded symptom-chasing. Second, automated verification is not optional in an AI-driven workflow, it is the mechanism that converts fast-but-fallible generation into trustworthy progress; the 90% coverage gates, mutation testing, and the three-job CI pipeline repeatedly caught issues that a review of the diff alone would have missed. Third, the human role shifts rather than shrinks: less time is spent typing boilerplate, and more is spent on architecture, scoping, review, and the interpretation of failures, activities that remain firmly a matter of engineering judgement.

These findings also have limits. They are drawn from a single project, a single primary agentic tool, and a compressed timeline; a different domain (e.g. one less represented in LLM training data than Spring Boot and React) or a longer-lived codebase might shift the balance between acceleration and rework. Nevertheless, within the scope of this lab, the disciplined human-in-the-loop workflow proved to be the decisive factor in making AI assistance a net gain.

10 Conclusion and Perspectives

This project delivered TimeTracker, a complete full-stack time-tracking application with hierarchical projects, sharing, tags, budgets, billable rates, JWT authentication, versioned database migrations, and a rigorously tested codebase enforced by a containerised CI pipeline. Just as importantly, it served as a controlled experiment in AI-driven software development: the entire implementation was carried out with an agentic coding tool operating under explicit human direction, review, and automated verification.

The main conclusion is methodological. AI tools were most effective when work was decomposed into small, well-specified units, when the agent was grounded in durable repository context rather than ad-hoc chat instructions, and when every change had to pass through tests and CI before acceptance. Conversely, the CI stabilisation episode showed that AI does not replace systems-level reasoning: understanding how a self-hosted runner, containers, service networking, and browser sandboxing interact required human diagnosis strategy, with the AI acting as an accelerator once the problem was properly framed.

Several perspectives follow for future work. On the product side, natural extensions include reporting dashboards with charts, calendar integrations, idle-time detection, and multi-currency support for billable rates. On the methodology side, it would be valuable to formalise the prompting patterns that emerged here, evidence-first debugging requests, scope-restricted tasks, and root-cause-before-edit constraints, into reusable agent instructions, and to evaluate them across multiple projects. Finally, the "missing tool" identified in this report, a sandboxed replica of the CI runtime in which proposed workflow fixes can be validated before pushing, points to a concrete direction in which agentic development environments could evolve, closing the last slow feedback loop that this project encountered.

11 References

1. Anthropic. (2025). *Claude Code: Agentic coding in your terminal* – Documentation. https://docs.anthropic.com/en/docs/claude-code
2. Spring Team. (2025). *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/
3. JetBrains. (2025). *Kotlin Language Documentation*. https://kotlinlang.org/docs/
4. Meta Open Source. (2025). *React Documentation*. https://react.dev/
5. Microsoft. (2025). *Playwright: Fast and reliable end-to-end testing for modern web apps*. https://playwright.dev/
6. PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation: WITH Queries (Common Table Expressions)*. https://www.postgresql.org/docs/current/queries-with.html
7. Redgate Software. (2025). *Flyway Documentation: Database migrations made easy*. https://documentation.red-gate.com/flyway
8. GitHub. (2025). *GitHub Actions Documentation*. https://docs.github.com/en/actions
9. Coles, H. (2025). *Pitest: Real world mutation testing for the JVM*. https://pitest.org/
10. Project repository: *TimeTracker – final-project-ayoubalila*. https://github.com/se2p-classrooms/final-project-ayoubalila